SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Name: A. Midhuna

Register No: 192521302

Course: Cryptography and Network Security

ASSIGNMENT 3-QUESTIONS

CO3: Examine cryptographic hash algorithms and digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication and non-repudiation.

Q1 – What Makes a Hash Function Trustworthy? (6 Marks)

Part A – Avalanche and Fixed-Length Digests

Shared Group Work

Together:

- Understand the required properties of a cryptographic hash (pre-image, second pre-image and collision resistance, avalanche effect)
- Implement a deliberately weak toy hash (for example, a byte-sum or XOR-fold hash) alongside a standard hash (SHA-256 from a library, used for comparison only)
- Decide how "avalanche" will be measured
- Run and test the experiment

Student A – Design & Mathematical Analysis

Student A must individually:

- Explain what each property means as an attack scenari
- Derive the expected number of output bits that change when one input bit is flipped in an ideal hash
- Explain mathematically why the toy hash fails the avalanche and second pre-image properties
- Explain why a fixed-length digest must have collisions

Student B – Experimental Validation

Student B must individually:

- Design the experimental inputs (single-bit flips, similar messages, long and short messages)
- Instrument the code to count the number of changed output bits
- Test at least three message sizes and a minimum number of trials per size
- Compare predicted and observed bit-change averages
- Explain any discrepancy

Part B – Collision Effort and the Birthday Bound

Shared Group Work

Together, build a collision finder for a truncated hash (take only the first k bits of SHA-256, for k = 16, 20, 24, 28, 32).

Student A – Experimental Investigation

Student A must:

- Design the collision-search runs and repeat each several times
- Record the number of hashes computed before the first collision
- Investigate increasing values of k
- Present the experimental evidence in a table and graph

Student B – Birthday-Bound Reasoning

Student B must:

- Derive why the expected effort is roughly 2^(k/2), not 2^k
- Explain the difference between finding any collision and finding a collision with a specific message
- Explain how this determines the minimum safe digest length
- Relate the result to the retirement of MD5 and SHA-1

Individual Verification

Faculty may ask Student A to explain the birthday bound and Student B to explain the avalanche experiment. Primary responsibility does not mean exclusive knowledge.

Answer:

Part A – Avalanche and Fixed-Length Digests
Properties of a Cryptographic Hash Function

- A trustworthy cryptographic hash function must have the following properties:
- Pre-image Resistance: It should be difficult to find the original message from its hash value.

  Attack scenario: An attacker tries to recover a password using its hash.
- Second Pre-image Resistance: It should be difficult to find another message that produces the same hash as a given message.

  Attack scenario: An attacker changes a document without changing its hash.
- Collision Resistance: It should be difficult to find two different messages that produce the same hash value.

  Attack scenario: An attacker creates two files with identical hash values to cheat a digital verification system.
- Avalanche Effect: A small change in the input should produce a large change in the output hash.

Expected Number of Changed Bits

SHA-256 produces a 256-bit hash.

When one input bit is flipped, each output bit has approximately a 50% probability of changing.

Expected changed bits:

\ [256 \times \frac {1}{2} =128 \]

Therefore, approximately 128 bits should change when one input bit is modified.

Toy Hash and SHA-256 Comparison

A toy hash can be created using byte-sum or XOR operations.

- Byte-sum hash: Adds all input bytes together.
- XOR-fold hash: Combines input bytes using XOR.
- SHA-256: Produces a fixed 256-bit digest.

The toy hash does not provide strong security because small input changes may produce only small output changes.

Why Toy Hash Fails

- It fails the avalanche effect because a small input change may cause only a small output change.
- It fails second pre-image resistance because different inputs can easily produce the same result.
- It is predictable and unsuitable for secure applications.

Why Fixed-Length Digests Have Collisions

A hash function converts unlimited possible input messages into a fixed-length output.

According to the pigeonhole principle, multiple inputs must eventually produce the same hash value.

Therefore, collisions are mathematically unavoidable.

Part B – Collision Effort and the Birthday Bound

Collision Search

A collision occurs when two different messages produce the same hash value.

For this experiment, SHA-256 is truncated to different bit lengths:

- 16 bits
- 20 bits
- 24 bits
- 28 bits
- 32 bits

As the number of bits increases, finding a collision becomes more difficult.

Birthday Bound

The birthday paradox explains why collisions occur earlier than expected.

The approximate number of hashes required to find the first collision is:

\ [N \approx 2^{k/2} \]

Where:

- N = Number of hashes required.
- k = Number of hash bits.

Expected Collision Effort

Hash bits (k)	Expected hashes
16	256
20	1,024
24	4,096
28	16,384
32	65,536

The table shows that collision effort increases as the hash length increases.

Any Collision vs Specific Message Collision

- Any collision: Finding any two different messages with the same hash.
- Specific message collision (second pre-image): Finding another message that matches the hash of a particular given message.

Finding any collision is generally easier than finding a second pre-image.

Minimum Safe Digest Length

A longer digest provides better collision resistance.

For an ideal k-bit hash, collision resistance is approximately \(2^{k/2}\).

Therefore, modern cryptographic applications require sufficiently long digests, such as SHA-256 or SHA-512.

Retirement of MD5 and SHA-1

- MD5: Produces a 128-bit hash and has practical collision attacks.
- SHA-1: Produces a 160-bit hash and is also vulnerable to practical collision attacks.
- SHA-256: Provides stronger security and is widely used today.

Therefore, MD5 and SHA-1 are not recommended for security-sensitive applications.

Conclusion

A trustworthy hash function should provide pre-image resistance, second pre-image resistance, collision resistance and a strong avalanche effect. SHA-256 satisfies these requirements much better than simple toy hash functions. The birthday bound explains why longer hash lengths are important for security.

Q2 – Integrity Is Not the Same as Authentication: Hash, MAC and HMAC (7 Marks)

Problem Context

A software distributor publishes a file together with its SHA-256 digest on the same web page. Another company protects internal messages with the construction H(key || message).

Shared Group Work

The pair must:

- Implement a simple Merkle–Damgård style toy hash (fixed block size, chaining value, padding with length)
- Implement the naive MAC H(key || message) and the HMAC construction
- Design tamper scenarios (modified message, modified message and recomputed digest, appended data)
- Conduct the experiments

Student A – Attack and Construction Investigation

Student A is individually responsible for:

- Showing that an attacker who can change both the file and the published digest defeats a plain hash
- Demonstrating a length-extension attack on the naive H(key || message) using your toy Merkle–Damgård hash
- Recording what the attacker needs to know and what the attacker does not need to know
- Explaining which property of the chaining construction makes the attack possible

Student B – Comparative Security Analysis

Student B is individually responsible for:

•Explaining the structure of HMAC (inner and outer hash, key padding) and why it resists length extension

- Comparing hash-only, naive MAC and HMAC in a table: which provides integrity, which provides authentication, and which provides neither
- Explaining why a MAC cannot provide non-repudiation
- Verifying through tests that HMAC detects every tamper scenario you designed

Joint Design Question

Together answer:

If you were designing the download-verification and message-protection components for an organisation, which mechanism would you choose for each, what must be kept secret, and what residual risk remains?

Both students must be able to defend the answer.

Answer:

Integrity Is Not the Same as Authentication: Hash, MAC and HMAC

Introduction

Hash, MAC and HMAC are security techniques used to protect data.

- Hash: Checks whether data has been modified.
- MAC: Provides message integrity and authentication using a secret key.
- HMAC: A secure method of generating a message authentication code using a cryptographic hash function and a secret key.

Merkle–Damgård Toy Hash

Merkle–Damgård is a hash construction that processes messages in fixed-size blocks.

It contains:

- Fixed block size: Divides the message into equal-sized blocks.
- Chaining value: Each block updates the previous hash value.
- Padding: Adds extra bits to complete the final block.
- Length padding: Includes the original message length.

The final chaining value becomes the hash digest.

Naive MAC – H(key || message)

A naive MAC is created by combining a secret key with a message and applying a hash function.

\[MAC=H(key||message) \]

Here:

- Key is secret.
- Message is the original data.
- Hash generates the authentication value.

It provides message integrity and authentication, but Merkle–Damgård based naive MAC is vulnerable to length-extension attacks.

HMAC Construction

HMAC stands for Hash-based Message Authentication Code.

Its formula is:

\ [HMAC=H ((K'\oplus opad) ||H ((K'\oplus ipad) ||M)) \]

Where:

- K' = Processed secret key.
- ipad = Inner padding.
- opad = Outer padding.
- M = Original message.
- H = Cryptographic hash function.

Why HMAC is secure:

- Uses two stages of hashing.
- Uses a secret key.
- Prevents attackers from directly extending the inner hash.
- Provides message integrity and authentication.

Attack Scenarios

Attack 1: Modified Message

An attacker changes the original file or message.

Result: The hash or authentication value changes, so the modification can be detected if the reference value is trusted.

Attack 2: Modified Message and Recomputed Digest

An attacker modifies both the file and its published SHA-256 digest.

Result: The receiver calculates the new hash and finds that it matches the modified published digest.

Therefore, a plain hash cannot prove who published the file.

Attack 3: Length-Extension Attack

An attacker uses the known hash value of a message and appends additional data to create a new valid hash for an extended message under a vulnerable naive MAC construction.

The attacker does not need to know the secret key.

Reason: Merkle–Damgård hashing exposes the final chaining value, which can be used as the starting state for processing additional blocks.

HMAC prevents this attack through its inner and outer hashing structure.

Comparison of Hash, MAC and HMAC

Feature	Hash	Naive MAC	HMAC
Integrity	Yes, with trusted digest	Yes	Yes
Authentication	No	Yes	Yes
Secret key	No	Yes	Yes
Length-extension resistance	Not applicable as a MAC	No	Yes
Security	Depends on hash	Weak construction	Strong

Why MAC Cannot Provide Non-Repudiation

A MAC uses a shared secret key between sender and receiver.

Both parties know the same secret key and can generate a valid MAC.

Therefore, the receiver cannot prove that only the sender created the message.

Digital signatures are used to provide non-repudiation because they use private and public keys.

Experimental Verification

The following tests can be conducted:

Test case	Hash	Naive MAC	HMAC
Original message	Pass	Pass	Pass
Modified message	Detects change if digest is trusted	Detects change	Detects change

Modified message with recomputed public digest	Cannot detect	Cannot forge without key	Cannot forge without key
Appended data	New digest required	Vulnerable to length extension	Detects tampering

Joint Design Question

For an organisation, I would choose:

Download Verification:

- Use SHA-256 to verify downloaded files.
- Publish the digest through an authenticated channel or digitally sign the digest.
- Keep the signing private key secret.

Internal Message Protection:

- Use HMAC-SHA-256.
- Keep the shared secret key confidential.
- Only authorised users should access the key.

Residual Risks:

- Secret key theft.
- Compromised systems.
- Incorrect key management.
- Attackers replacing both the file and its unauthenticated digest**.**

Conclusion

A plain hash provides data integrity checking but does not provide authentication. A naive MAC provides authentication but is vulnerable to length-extension attacks when used with Merkle–Damgård hashing. HMAC provides stronger security through inner and outer hashing. Therefore, SHA-256 with authenticated distribution is suitable for file verification, and HMAC-SHA-256 is suitable for protecting internal messages.

Q3 – Digital Signatures: Hash-then-Sign and What Can Break Them (7 Marks)

Shared Group Work

Together:

- Implement RSA signature generation and verification using hash-then-sign

(small keys for measurement)
- Implement a DSA-style (or ECDSA-style) signature using toy parameters
- Test valid signatures, modified messages, modified signatures and wrong public keys
- Conduct the nonce experiment described below

Student A – Algorithmic Derivation

Student A must individually:

- Explain why the message is hashed before signing and what would go wrong with signing the raw message
- Prove why verification succeeds for a correctly generated RSA signature ($s = H(m)^d \bmod n$ and $s^e \equiv H(m) \bmod n$)
- Derive the DSA/ECDSA verification equation and show why it recovers the signer's value
- Explain what security goal each step contributes (integrity, authentication, non-repudiation)

Student B – Attack Experiment and Evidence

Student B must individually:

- Design the test messages and tamper cases and record accept/reject results
- Demonstrate nonce reuse in DSA/ECDSA: sign two messages with the same per-signature secret k, then recover k and the private key from the two signatures
- Demonstrate a textbook (no hash) RSA signature forgery (for example, multiplicative property: signature of $m_1 \cdot m_2$ from signatures of m_1 and m_2)
- Explain what these results show about "correct maths but unsafe implementation"

Joint Conclusion

Together answer:

A valid signature proves that someone holding the private key signed the message. Under what conditions does this actually amount to non-repudiation,

and what could the signer claim to deny it?

Consider key compromise, timestamping, and the role of a trusted third party.

Answer:

Digital Signatures: Hash-then-Sign and What Can Break Them

Introduction

A digital signature is a security technique used to verify the authenticity and integrity of digital messages or documents.

It uses two keys:

- Private Key: Used to create the digital signature.
- Public Key: Used to verify the digital signature.

Common digital signature algorithms include RSA, DSA and ECDSA.

Why Hash Before Signing?

In digital signatures, the message is first converted into a fixed-length hash value and then signed.

$$h=H(m)$$

$$s=h^d \bmod n$$

Reasons:

1. Reduces the amount of data to be signed.
2. Produces a fixed-length digest.
3. Helps detect message modifications.
4. Improves signing efficiency.

Signing raw messages directly can be inefficient and may allow mathematical attacks if insecure textbook RSA is used.

RSA Signature Generation and Verification

Signature Generation:

$$s=H(m)^d \bmod n$$

Where:

- m = Original message.
- H(m) = Message hash.
- d = Private exponent.
- n = RSA modulus.
- s = Digital signature.

Signature Verification:

$$ s^e \bmod n=H(m) $$

Since RSA satisfies:

$$ ed\equiv1\pmod{\phi(n)} $$

Therefore,

$$ s^e=(H(m)^d)^e $$

$$ =H(m)^{de}\bmod n $$

$$ =H(m) $$

Hence, verification succeeds for a correctly generated signature, assuming the hash value is represented correctly modulo n.

DSA/ECDSA Signature Verification

DSA uses a private key and a public key.

The signature generation equations are:

$$ r=(g^k\bmod p)\bmod q $$

$$ s=k^{-1}(H(m)+xr)\bmod q $$

Where:

- k = Secret nonce.
- x = Private key.
- r and s = Signature values.

Verification:

$$ w=s^{-1}\bmod q $$

$$ u_1=H(m)w\bmod q $$

$$ u_2=rw\bmod q $$

$$ v=(g^{u_1}y^{u_2}\bmod p)\bmod q $$

The signature is accepted when:

$$ v=r $$

This verifies that the signature corresponds to the message and the public key.

Security Goals of Digital Signatures

- Integrity: Ensures the message has not been modified.
- Authentication: Confirms that the signature corresponds to the private key associated with the public key.
- Non-repudiation: Provides evidence that the private key was used to sign the message, subject to proper key ownership and security.

Experimental Test Cases

Test case	Expected result

Original message and valid signature	Accept
Modified message	Reject
Modified signature	Reject
Wrong public key	Reject
Invalid signature	Reject

These experiments demonstrate that digital signatures detect message and signature modifications.

Nonce Reuse Attack in DSA/ECDSA

A nonce is a secret random number used during signature generation.

If the same nonce k is reused for two different messages, the private key can be recovered.

For two signatures:

$$ s_1=k^{-1}(H(m_1)+xr)\bmod q $$

$$ s_2=k^{-1}(H(m_2)+xr)\bmod q $$

Subtracting:

$$ k=(H(m_1)-H(m_2))(s_1-s_2)^{-1}\bmod q $$

After recovering k:

$$ x=(s_1k-H(m_1))r^{-1}\bmod q $$

Therefore, the attacker can recover the private key.

Result: Reusing the nonce can completely compromise the digital signature system.

Textbook RSA Signature Forgery

Textbook RSA has a multiplicative property.

If:

$$ s_1^e=m_1\bmod n $$

$$ s_2^e=m_2\bmod n $$

Then:

$$ (s_1s_2)^e=m_1m_2\bmod n $$

Therefore, an attacker can create a signature for the product of two messages without knowing the private key.

This is why secure RSA signatures use hashing and padding schemes such as RSA-PSS.

What Does Correct Mathematics but Unsafe Implementation Mean?

Even when an algorithm is mathematically correct, poor implementation can cause security problems.

Examples:

- Reusing secret nonces.
- Using weak keys.
- Using textbook RSA without secure padding.

- Exposing private keys.
- Using weak random number generators.

Therefore, secure implementation is as important as correct mathematical design.

Joint Conclusion – Non-Repudiation

A valid digital signature can provide evidence that a message was signed using a particular private key.

However, non-repudiation depends on the following conditions:

- The private key must remain confidential.
- The public key must be correctly linked to the signer through a trusted digital certificate.
- The signing time should be reliably recorded using trusted timestamping.
- A trusted third party, such as a Certificate Authority (CA), should verify the identity of the signer.

What could the signer claim to deny it?

- The private key was stolen or compromised.
- Someone else used the signing device.
- The certificate was incorrectly issued.
- The signature was created after the key was compromised.

Final Result:

Digital signatures provide integrity, authentication and evidence of signing. However, nonce reuse, weak padding, key compromise and poor implementation can break their security. Therefore, secure algorithms, proper key management and trusted timestamping are essential for reliable non-repudiation.

Q4 – Trust Through Certificates: X.509 Chain Validation (7 Marks)

Here the responsibilities are deliberately reversed so that the same student does not always perform the theoretical component.

Shared Group Work

Together:

- Study the structure of an X.509 v3 certificate (subject, issuer, serial number, validity, public key, extensions such as key usage and basic constraints, signature)

- Build a small three-level hierarchy (root CA → intermediate CA → end-entity) using a library or a toolkit such as OpenSSL
- Implement a chain-validation procedure that follows the steps a client must perform
- Verify the procedure on valid and invalid chains

Student A – Failure-Case Experiments

Student A must individually:

- Design at least six failure cases: expired certificate, not-yet-valid certificate, wrong issuer, tampered certificate field, hostname mismatch, end-entity certificate used as a CA, revoked certificate
- Record which step of validation rejects each case
- Compare CRL and OCSP (freshness, size, privacy, availability) and show a case in which revocation is detected only if the check is actually performed
- Discuss what a "fail-open" revocation policy means for security

Student B – Trust Model & Validation Logic

Student B must individually:

- Write the complete validation algorithm in pseudocode (signature check on each link, issuer–subject matching, validity period, basic-constraints CA flag, key usage, name matching, revocation check, anchoring at a trusted root)
- Explain what each step protects against
- Explain why trust in the root cannot itself be verified cryptographically
- Explain how X.509 binds an identity to a public key and why that binding is what makes a signature meaningful

Joint Engineering Decision

Together determine

If a CA is compromised or a certificate is mis-issued, what limits the damage, how long can the damage persist, and what would you change in a deployment to shorten that window?

Answer:

Trust Through Certificates: X.509 Chain Validation (7 Marks)

Introduction

X.509 is a digital certificate standard used to verify the identity of websites, servers and users.

It connects an identity with a public key using a digital certificate issued by a trusted Certificate Authority (CA).

- A certificate chain contains three levels:
- Root CA
- Trusted Certificate Authority
- Intermediate CA
- Issues end-entity certificates
- End-Entity Certificate

Structure of X.509 v3 Certificate

- An X.509 certificate contains the following information:
- Subject: Identity of the certificate owner.
- Issuer: Name of the CA that issued the certificate.
- Serial Number: Unique certificate identification number.
- Validity: Starting and expiration dates.
- Public Key: Public key of the certificate owner.
- Extensions: Key usage, basic constraints and subject alternative names.
- Digital Signature: Verifies that the certificate was signed by the issuing CA.

Three-Level Certificate Hierarchy

The certificate hierarchy contains:

- Root CA: Main trusted authority.
- Intermediate CA: Receives authority from the root CA.
- End-Entity: Represents the website or server.

The root CA signs the intermediate certificate, and the intermediate CA signs the end-entity certificate.

The client verifies the complete certificate chain before trusting the website.

Certificate Validation Procedure

The client performs the following steps:

- Check the certificate validity period.
- Verify the issuer and subject names.
- Verify the digital signature of each certificate.

- Check the CA basic constraints.
- Check key usage permissions.
- Verify the website hostname.
- Check certificate revocation status.
- Confirm that the chain ends at a trusted root CA.

If all checks succeed, the certificate chain is accepted.

Student A – Failure-Case Experiments

The following failure cases are tested:

Failure case	**Expected rejection reason**
Expired certificate	Validity check fails
Not-yet-valid certificate	Validity check fails
Wrong issuer	Issuer matching fails
Tampered certificate	Signature verification fails
Hostname mismatch	Name matching fails
End-entity used as CA	Basic constraints fail
Revoked certificate	Revocation check fails

CRL and OCSP Comparison

CRL stands for Certificate Revocation List.

OCSP stands for Online Certificate Status Protocol.

Feature	**CRL**	**OCSP**
Method	Downloads revocation list	Checks certificate status online
Freshness	Updated periodically	Usually provides current status
Size	Can be large	Smaller response
Privacy	Less privacy	CA may know which certificate is checked
Availability	Can use downloaded list	Requires OCSP service availability

Revocation experiment:

If a certificate is revoked but the client does not perform a revocation check, it may still accept the certificate if all other checks succeed. Fail-open policy: If the revocation service is unavailable, the client accepts the certificate anyway. This can create security risks because revoked certificates may be trusted.

Why Root Trust Cannot Be Verified Cryptographically

A root CA is usually self-signed. Its signature can be mathematically verified using its own public key, but this does not prove that the root CA is trustworthy.

Trust must be established through an external process, such as:

- Operating system trust stores.
- Browser trust stores.
- Organisation security policies.

Therefore, root trust depends on the user's or organisation's decision to trust that CA.

Purpose of Each Step

Validation step	Security purpose
Validity period	Prevents expired certificate usage
Issuer matching	Checks certificate hierarchy
Signature verification	Detects certificate modification
Basic constraints	Prevents unauthorised CA usage
Key usage	Restricts certificate operations
Hostname matching	Confirms website identity
Revocation check	Detects compromised certificates
Trusted root	Establishes trust

How X.509 Connects Identity and Public Key

X.509 certificates connect an individual's or website's identity with a public key.

A trusted CA verifies the identity and signs the certificate. When a client verifies the CA's signature, it gains confidence that the public key belongs to the stated identity. This makes digital signature verification meaningful because the client can associate the verified public key with the correct website or organisation.

Joint Engineering Decision

If a CA is compromised or issues an incorrect certificate, the following measures help limit the damage.

Damage Limitation:

- Revoke compromised certificates.
- Remove compromised CA certificates from trusted stores.
- Use intermediate CAs instead of directly issuing certificates from the root.
- Monitor certificate transparency logs.
- Use short certificate validity periods.

- How long can damage persist?

The damage may continue until the certificate expires, revocation is detected, or the compromised CA is removed from trust stores.

Improvements:

- Use short-lived certificates.
- Perform regular OCSP or CRL checks.
- Protect CA private keys using Hardware Security Modules (HSMs).
- Monitor certificate issuance.
- Update trusted root certificates regularly.

Conclusion

X.509 certificate validation establishes trust between websites and users by connecting identities with public keys. Certificate chain validation checks signatures, validity periods, issuer information, hostname and revocation status. Therefore, proper CA management, revocation checking and secure key storage are essential for maintaining certificate security.

Q5 – Kerberos: Authentication Without Sending Passwords (6 Marks)

Shared Group Work

Together:

Study the Kerberos message flow (Authentication Server exchange, Ticket-Granting Server exchange, Client–Server exchange)

- Build a simulation with a client, KDC (AS + TGS) and one service, using symmetric encryption and authenticated tickets
- Include timestamps, ticket lifetimes, session keys and authenticators
- Test the normal flow and the attack/failure cases below

Student A – Protocol Design

Student A must individually:

- Draw the message-sequence diagram and label what is encrypted with which key
- Explain what the Ticket-Granting Ticket is for and why the user's long-term key is used only once per session
- Explain the purpose of the authenticator and the timestamp
- Explain how mutual authentication is achieved
- Identify which party can read which part of each message

Student B – Attack and Failure Evidence

Student B must individually:

- Demonstrate a replay of a captured authenticator with and without replay protection
- Demonstrate an expired ticket and a clock-skew failure
- Demonstrate what happens if the KDC is unavailable
- Record accept/reject outcomes and the reason for each
- Explain what a stolen ticket or an offline password-guessing attack on the AS reply would allow

Cross-Understanding Requirement

Student B must be able to explain:
Why does the client never send its password across the network, and what does the service learn about the client?

Student A must be able to explain:
What experimental evidence shows that the timestamp/authenticator mechanism actually prevents replay?This deliberately tests understanding across the assigned responsibilities.

Answer:

Kerberos: Authentication Without Sending Passwords (6 Marks)

Introduction

Kerberos is a network authentication protocol that verifies the identity of users without sending their passwords across the network.

It uses symmetric key encryption, tickets, timestamps and session keys to provide secure authentication.

The main components are:

- Client: User requesting access to a service.
- Authentication Server (AS): Verifies the user's identity.
- Ticket Granting Server (TGS): Provides service tickets.
- Service Server: Provides the requested service.
- KDC: Key Distribution Center, which contains AS and TGS.

Kerberos Message Flow

- Kerberos authentication consists of three exchanges.
- Authentication Server (AS) Exchange

- Client requests authentication.
- Client → AS: Username and request for TGT
- AS → Client: TGT and client–TGS session key
- Ticket Granting Server (TGS) Exchange

Client requests access to a service.

- Client → TGS: TGT, authenticator and service request
- TGS → Client: Service ticket and client–server session key
- Client–Server Exchange
- Client accesses the requested service.
- Client → Server: Service ticket and authenticator
- Server → Client: Encrypted timestamp confirmation
- Mutual authentication completed

Ticket-Granting Ticket (TGT)

A TGT is a ticket issued by the Authentication Server.

- It allows the client to request multiple service tickets.
- It avoids repeatedly entering the password.
- It is encrypted using the TGS secret key.
- Only the TGS can decrypt and read it.

The user's long-term key is used to protect the AS reply containing the client–TGS session key. It is not used for every service request.

Authenticator and Timestamp

Authenticator:

An authenticator contains the client's identity and timestamp. It is encrypted using the session key.

It proves that the client knows the session key and helps prevent replay attacks.

Timestamp:

- Confirms that the request is recent.
- Prevents reuse of old authentication messages.
- Helps detect replay attacks.
- Requires synchronised clocks between systems.

Mutual Authentication

Mutual authentication means both the client and server verify each other's identity.

- Client sends a service ticket and authenticator.
- Server verifies the ticket and authenticator.
- Server sends an encrypted timestamp response.
- Client verifies the response.

Thus, both parties confirm that they possess the correct session key.

Which Party Can Read Each Message?

Message	Who can read it?
Client → AS	AS reads the request
AS reply	Client decrypts its encrypted part; TGS reads the TGT
Client → TGS	TGS reads the ticket and authenticator
TGS reply	Client decrypts its reply; service reads its ticket
Client → Server	Server decrypts the ticket and authenticator
Server → Client	Client decrypts the response

The passwords and long-term secret keys are never directly transmitted**.**

Student B – Attack and Failure Experiments

Experiment 1: Replay Attack

An attacker captures an authenticator and sends it again.

- Without replay protection: The server may accept the repeated request.
- With replay protection: The server rejects the repeated timestamp/authenticator.

Result: Timestamp checking and replay caches prevent repeated authentication requests.

Experiment 2: Expired Ticket

An attacker attempts to use a ticket after its lifetime expires.

Result: Server rejects the expired ticket because its validity period has ended.

Experiment 3: Clock-Skew Attack

If the client and server clocks differ beyond the allowed time difference, authentication fails.

Result: Server rejects the request because the timestamp is outside the permitted clock-skew window.

Experiment 4: KDC Unavailable

If the KDC is unavailable:

- New authentication requests cannot be completed.
- New service tickets cannot be obtained.
- Previously issued valid tickets may continue working until they expire, depending on the service.

Result: Authentication of new sessions fails.

Experimental Results Table

Test case	Expected result	Reason
Valid authentication	Accept	Correct credentials and tickets
Replayed authenticator	Reject	Replay detected
Expired ticket	Reject	Ticket lifetime exceeded
Clock-skew failure	Reject	Invalid timestamp
KDC unavailable	Reject new authentication	KDC cannot issue tickets
Valid ticket and authenticator	Accept	Authentication successful

Stolen Ticket and Offline Password Guessing

Stolen Ticket:

If an attacker steals a valid service ticket, they may impersonate the client until the ticket expires, provided they can also satisfy the authenticator requirements or exploit the stolen credentials/session key.

Offline Password Guessing:

An attacker who captures certain AS replies may try different password guesses offline to recover the user's password-derived key.

A weak password makes this attack easier.

Strong passwords and suitable encryption methods reduce this risk.

Cross-Understanding Questions

Why does the client never send its password across the network?

The password is used to derive a secret key locally. Kerberos uses encrypted session keys and tickets instead of transmitting the actual password.

What does the service learn about the client?

The service learns the client's authenticated identity, ticket validity and the session key used for secure communication. It does not learn the client's password.

What evidence proves replay protection works?

The experiment shows that the first valid authenticator is accepted, but sending the same authenticator again is rejected because its timestamp and identity have already been recorded.

Conclusion

Kerberos provides secure network authentication without transmitting passwords. It uses AS, TGS, tickets, session keys and timestamps to authenticate users and services. Replay protection, ticket expiration and mutual authentication improve security. Therefore, Kerberos is widely useful for secure authentication in organisational networks.

Q6 – Advanced Security Mini-Investigations: RSA, DSA/DSS, ECDSA, ElGamal and Schnorr (7 Marks)

This question provides particularly clear individual evidence.
Student A – Hash Weakness and Signature Forgery
Student A takes primary responsibility for the collision-to-forgery investigation.

The student must:

- Explain the difference between a collision attack and a second pre-image attack
- Using your truncated-hash collision finder from Q1, build two different messages with the same short digest and show that a signature on one is valid for the other (use a signature scheme over the truncated digest)
- Explain why this attack requires the attacker to prepare both messages in advance
- Explain how a collision in MD5/SHA-1 enabled real-world forged certificates and what this means for X.509
- Explain the role of digest length and algorithm choice in deciding what a CA may use
- Recommend a hash policy for signatures and justify it

Student B Verification

Student B must provide a short explanation of

Why does weakening the hash function weaken the signature, even when the signature algorithm itself (RSA/DSA) is unchanged?

Student B – Kerberos versus X.509 and Non-Repudiation

Student B takes primary responsibility for the framework comparison investigation.

The student must:

- Compare Kerberos and X.509/PKI in a table: key type, trusted third party, online/offline dependence, scalability, revocation, typical environment
- Explain which framework gives non-repudiation and which does not, and why (shared secret vs private key held by one party)
- Analyse the single point of failure of Kerberos and the trust-anchor risk in

X.509

- Compare the number of key relationships required in a network of n users for pairwise shared keys, Kerberos with a KDC, and PKI
- Recommend which framework you would use for (i) a campus single sign-on system and (ii) signing legally binding electronic contracts

Student A Verification

Student A must provide a short explanation of:

Why can a Kerberos ticket prove who a user is to a service, yet not prove to a third party later that the user performed an action?

Thus, Q6 provides different primary tasks but mandatory cross-understanding.

Evidence Expected: worked toy example, verification trace, attack/failure cases and a comparison table covering RSA, DSA/DSS, ElGamal and Schnorr.

• **Comparison requirement**: Compare RSA, DSA/DSS, ElGamal and Schnorr signatures with respect to key material, randomness/nonce requirements, verification, forgery resistance and non-repudiation assumptions.

• **Schnorr protocol**: Describe the commitment–challenge–response sequence, derive the verification relation for toy parameters, and explain why the challenge must be unpredictable and why nonce reuse is dangerous.

• **ElGamal signature scheme**: Derive key generation, signing and verification for small toy parameters; test a valid signature and at least two altered-message/signature cases; explain how poor handling of signing randomness can weaken security.

Answer:

Student A: Hash Weakness and Signature Forgery

Collision Attack vs Second Pre-image Attack

Collision Attack	Second Pre-image Attack
Finds any two different messages with the same hash.	Finds another message matching a given message's hash.
Attacker can choose both messages.	One message is already fixed.
Easier to perform.	Generally harder to perform.

Collision-to-Signature Forgery

A truncated hash produces a shorter digest by taking only the first few bits of SHA-256.

For example, consider:

- Message 1: I approve the payment of Rs.100
- Message 2: I approve the payment of Rs.10000

Suppose both messages produce the same 16-bit digest.

\ [H(M1) =H(M_2) \]

The attacker obtains a signature for Message 1.

\ [S= Sign (PrivateKey,H(M_1)) \]

Since both messages have the same digest:

\ [H(M_1) =H(M_2) \]

The same signature also verifies for Message 2.

Result: A signature intended for Message 1 can be misused to authenticate Message 2.

Why Must Both Messages Be Prepared in Advance?

The attacker must prepare both messages before obtaining the signature.

This is because the attacker needs to create two different messages with identical hash values and then convince the signer to sign the harmless message.

After receiving the signature, the attacker substitutes the harmful message.

MD5 and SHA-1 Certificate Forgery

MD5 and SHA-1 have known collision vulnerabilities. Attackers demonstrated that collisions could be used to create forged digital certificates. This creates risks for X.509 because certificate signatures depend on secure hash functions. If an attacker creates two certificates with the same digest, a signature on one may be misused for the other.

Role of Digest Length and Algorithm Choice

A longer digest provides better collision resistance.

For an ideal k-bit hash, collision effort is approximately:

\[2^{k/2} \]

Therefore:

- MD5 – 128 bits: Insecure.
- SHA-1 – 160 bits: Insecure.
- SHA-256 – 256 bits: Recommended.
- SHA-512 – 512 bits: Strong security.

Recommended Hash Policy

I recommend SHA-256 or SHA-384 for digital signatures.

Reasons:

1. Strong collision resistance.

2. Fixed-length output.
3. Widely supported.
4. Suitable for modern digital signature algorithms.

For RSA signatures, RSA-PSS should be preferred over textbook RSA.

Student B: Kerberos vs X.509 and Non-Repudiation

Comparison Between Kerberos and X.509

Feature	Kerberos	X.509/PKI
Key type	Symmetric keys	Public and private keys
Trusted third party	KDC	Certificate Authority
Online dependence	KDC needed for new tickets	CA not needed for every signature verification
Scalability	Good for organisations	Suitable for large networks
Revocation	Ticket expiration and account/key controls	CRL and OCSP
Environment	Campus and enterprise networks	Websites, digital signatures and secure communication
Non-repudiation	No	Can support it

Non-Repudiation Difference

- Kerberos uses shared secret keys between the parties.
- Both parties may possess the same key and generate valid authentication messages.
- Therefore, it cannot provide strong non-repudiation.
- X.509/PKI:
- Digital signatures use a private key held by the signer and a public key used for verification.
- This provides stronger evidence that the corresponding private key was used.
- With secure key management and identity verification, it can support non-repudiation.

Single Point of Failure and Trust Risks

- The KDC is a central component.
- If the KDC becomes unavailable, new authentication requests cannot be processed. If its database is compromised, attackers may obtain sensitive keys.
- X.509:
- The root CA is a major trust anchor.

- If the root CA is compromised, attackers may issue fraudulent certificates that appear trusted.

Recommended Security Framework

(i) Campus Single Sign-On System:

I recommend Kerberos because:

- Centralised authentication.
- Secure ticket-based access.
- Passwords are not transmitted.
- Suitable for campus networks.

(ii) Legally Binding Electronic Contracts:

I recommend X.509/PKI digital signatures because:

- Provides signer identity verification.
- Supports digital signature verification.
- Can provide evidence of signing.
- Supports trusted timestamping.

Commitment–Challenge–Response

Schnorr is a digital signature scheme based on discrete logarithm problems.

It contains three main stages:

1. Commitment: Signer selects a random nonce and calculates a commitment.
2. Challenge: A challenge value is generated using a hash function.
3. Response: Signer calculates the response using the private key and nonce.

Schnorr Mathematical Equations

Key generation:

$$ y=g^x\bmod p $$

Commitment:

$$ R=g^k\bmod p $$

Challenge:

$$ e=H(R||m) $$

Response:

$$ s=k+ex\bmod q $$

Verification:

$$ g^s=R\cdot y^e\bmod p $$

Why Is the Challenge Important?

The challenge must be unpredictable and securely generated.

If the same nonce is reused for different messages, the private key can be recovered.

Therefore, nonce generation must be secure.

Conclusion

RSA, DSA, ECDSA, ElGamal and Schnorr provide digital signature mechanisms for authentication and integrity. However, weak hash functions, nonce reuse and poor key management can compromise their security. Kerberos is suitable for campus authentication, while X.509/PKI is suitable for digital signatures and electronic contracts. Secure algorithms and proper key management are essential for trustworthy digital communication.

Q7 – Design a Secure Authentication and Integrity Scheme from First Principles (10 Marks)

Problem Context – Secure Over-the-Air Update for Medical Devices

A manufacturer distributes firmware updates to thousands of infusion pumps in hospitals. Each pump must be sure that an update (a) has not been altered, (b) really comes from the manufacturer, and (c) the manufacturer cannot later deny having issued it. Pumps have limited memory, are often offline, and have unreliable clocks. Some pumps must also authenticate to the hospital's central server before accepting a service session.

Stage 1 – Independent Design Proposal

Before discussing the solution with each other:

Student A independently proposes:

- Which security goals each mechanism will address
- Which primitives (hash, MAC, signature, certificate, Kerberos-style ticket) will be used and where
- What key or certificate material each party holds
- A possible message flow for one update

Student B independently proposes:

- Which security goals each mechanism will address
- Which primitives (hash, MAC, signature, certificate, Kerberos-style ticket) will be used andwhere
- What key or certificate material each party holds
- A possible message flow for one update

Each student's initial proposal must be retained in the submission. No marks are deducted merely because an initial design is incorrect. The purpose is to provide evidence of independent design thinking.

Stage 2 – Compare the Two Designs

The pair must now compare the two independent proposals. Discuss:

- Where the designs agree?
- Where they differ?
- Strengths and weaknesses of each
- Missing threats or missing cases (rollback to an old version, stolen signing key, offline pump, wrong clock, revoked key)
- Which ideas should be retained?

Evidence Expected: A short design-comparison record showing how the two initial ideas influenced the final solution.

Stage 3 – Joint Final Design

Using the strongest ideas from the discussion, jointly develop the final scheme. The final design must identify:

- Threat model (who the attacker is and what the attacker can do)
- Security goals and the mechanism that achieves each
- Key and certificate management (issuance, storage, rotation, revocation)
- Message flow / protocol (diagram) for signing, distributing and verifying an update
- Behaviour when the pump is offline or its clock is wrong
- Complete pseudocode for signing and for verification

Stage 4 – Divide the Validation Responsibilities

Student A – Security Argument Validation

Student A must:

- State what each attack would need to succeed (e.g., 2^(n/2) work for a collision, forging a signature, compromising a key)
- Show, goal by goal, why the final design prevents or limits each attack
- Estimate the attacker's effort for your chosen digest and key sizes using the results from Q1 and Q3
- Identify the residual risks that remain after the design

Student B – Experimental Validation

Student B must:

- Design correctness and attack test cases (valid update, modified image, modified signature, replayed older version, expired or revoked certificate, wrong key)
- Implement and instrument the final verification procedure
- Compare with a straightforward baseline (e.g., a bare SHA-256 digest published beside the file) to show which attacks the baseline fails to stop

- Record accept/reject outcomes, verification cost (hash operations, signature verifications) and the size of the added metadata

Stage 5 – Cross-Defence

Student A must explain:

What experimental evidence supports the group's design?

Student B must explain:

How does the security argument follow from the group's choice of mechanisms, and what does each mechanism contribute?

This ensures that both students understand both the theoretical and experimental dimensions.

Stage 6 – Joint Evaluation

Together determine:

- Which of integrity, authentication and non-repudiation is guaranteed by which part of the design
- What costs the design introduces (verification time, certificate storage, key-management burden)
- Under what conditions the design could still fail (key compromise, flawed implementation, compromised root, bad clock)
- Under what conditions a simpler or a different mechanism (e.g., HMAC with shared key, Kerberos tickets) would be more appropriate

Answer:

Stage 1 – Independent Design Proposal

Student A – Initial Design

- Use SHA-256 to check firmware integrity.
- Use RSA digital signatures to verify the manufacturer.
- Use X.509 certificates to verify the manufacturer's public key.
- Use a private key to sign firmware updates.
- Use a Kerberos-style ticket to authenticate pumps to the hospital server.

Message flow:

- Manufacturer creates firmware.
- SHA-256 generates the hash.
- Manufacturer signs the hash using its private key.
- Pump verifies the signature using the manufacturer's public key.
- Pump installs the firmware if verification succeeds.

Student B – Initial Design

- Use SHA-256 for data integrity.
- Use HMAC-SHA-256 for message authentication.
- Use digital certificates to verify the manufacturer.
- Use session keys for secure communication.
- Use timestamps and version numbers to prevent replay attacks.

Message flow:

- Manufacturer generates the firmware hash.
- Manufacturer creates an authentication value.
- Pump checks the hash and authentication value.
- Pump accepts the update if both checks succeed.

Compare the Two Designs

Comparison	Student A	Student B
Integrity	SHA-256	SHA-256
Authentication	RSA signature	HMAC
Certificate	X.509	Digital certificate
Replay protection	Version number	Timestamp and version
Main strength	Supports non-repudiation	Efficient authentication
Weakness	Certificate management required	Shared key cannot provide non-repudiation

Missing Threats

- Rollback to an older firmware version.
- Theft of manufacturer's private key.
- Offline pump verification.
- Incorrect device clock.
- Revoked certificates.
- Modification of firmware during distribution**.**

Final Decision

The group selects Student A's digital-signature approach because the manufacturer must provide non-repudiation. Student B's version-control and replay-protection ideas are also included.

Joint Final Design

1. Threat Model

Possible attackers include:

- Hackers modifying firmware files.
- Attackers distributing fake firmware.

- Attackers replaying older firmware versions.
- Attackers stealing cryptographic keys.
- Attackers modifying certificates.
- Attackers performing man-in-the-middle attacks.

The system must protect firmware updates against these attacks.

2. Security Goals and Mechanisms

Security goal	Mechanism
Integrity	SHA-256
Authentication	RSA digital signature
Non-repudiation	Manufacturer's digital signature
Identity verification	X.509 certificate
Replay protection	Firmware version number
Secure communication	Kerberos tickets
Key protection	Secure key storage

3. Key and Certificate Management

Manufacturer:

- Holds a private signing key.
- Uses an X.509 certificate.
- Signs firmware updates.

Medical Device (Pump):

- Stores the manufacturer's trusted root certificate.
- Stores the highest successfully installed firmware version.
- Uses the public key to verify signatures.

Hospital Server:

- Uses Kerberos authentication.
- Maintains a Key Distribution Center (KDC).
- Issues tickets for authorised service sessions.

Key Management:

- Private keys must be stored securely.
- Certificates must be renewed before expiration.
- Compromised certificates must be revoked.
- New keys must be securely distributed to devices.

4. Message Flow Diagram

Manufacturer

Creates firmware and calculates SHA-256

Digital Signature

Signs firmware hash and update metadata using private key

Distribution Server

Distributes firmware, signature and certificate chain

Infusion Pump

Checks certificate, signature, hash and firmware version

Accept and Install Update

Only if every security check succeeds

5. Offline and Incorrect Clock Behaviour

Offline Pump:

- The pump can verify firmware using its stored trusted root certificate.
- It does not need an internet connection to verify the signature.
- The update can be installed after all local checks succeed.

Incorrect Clock:

- The pump should not depend only on its local clock.
- It should verify signed update metadata and certificate validity using a trusted time source.
- If reliable time cannot be established, the pump should reject updates whose certificate validity cannot be confirmed.
- Digital Signatures:
- Suitable for medical firmware updates because the manufacturer must prove the authenticity of updates without sharing its private signing key with every pump.

Final Conclusion

The proposed secure firmware update system combines SHA-256, RSA digital signatures, X.509 certificates and version control. SHA-256 ensures integrity, RSA provides authentication and evidence of signing, and X.509 establishes trust in the manufacturer's identity. Kerberos is used separately for hospital service authentication. This combination provides strong protection against firmware modification, fake updates and rollback attacks while supporting offline verification.